

Extreme programming (XP) is one of the most important software development frameworks of Agile models. It is used to improve software quality and responsiveness to customer requirements. The extreme programming model recommends taking the best practices that have worked well in the past in program development projects to extreme levels.

What is Extreme Programming (XP)?

Extreme Programming (XP) is an Agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation. XP emphasizes a close working relationship between the development team, the customer, and stakeholders, with an emphasis on rapid, iterative development and deployment.

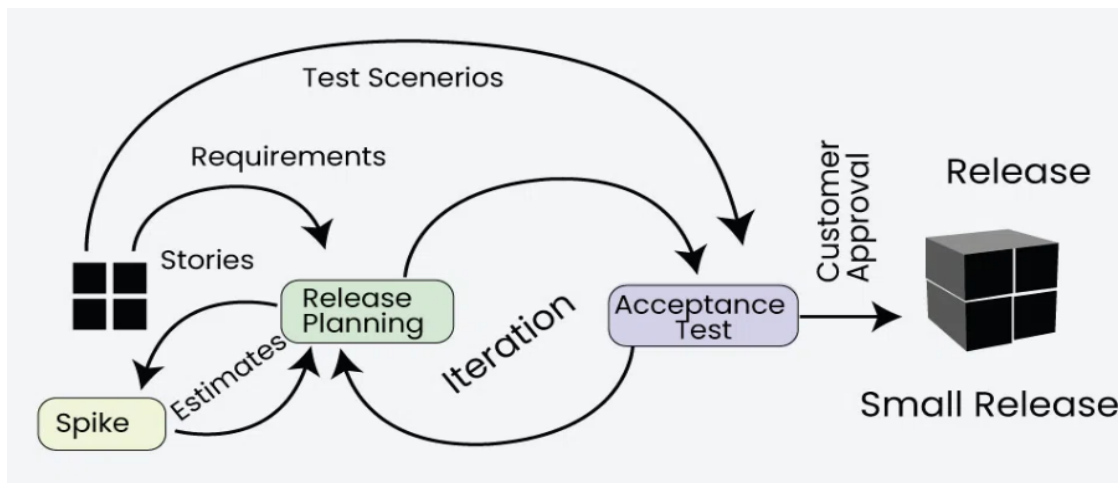


Fig1: Extreme Programming (XP)

Agile development approaches evolved in the 1990s as a reaction to documentation and bureaucracy-based processes, particularly the waterfall approach. Agile approaches are based on some common principles, some of which are:

1. Working software is the key measure of progress in a project.
2. For progress in a project, therefore software should be developed and delivered rapidly in small increments.
3. Even late changes in the requirements should be entertained.
4. Face-to-face communication is preferred over documentation.
5. Continuous feedback and involvement of customers are necessary for developing good-quality software.

6. A simple design that involves and improves with time is a better approach than doing an elaborate design up front for handling all possible scenarios.
7. The delivery dates are decided by empowered teams of talented individuals.

Extreme programming is one of the most popular and well-known approaches in the family of agile methods. an XP project starts with user stories which are short descriptions of what scenarios the customers and users would like the system to support. Each story is written on a separate card, so they can be flexibly grouped.

Basic Principles and Good Practices in Extreme Programming

XP is based on the frequent iteration through which the developers implement User Stories. User stories are simple and informal statements of the customer about the functionalities needed. A User Story is a conventional description by the user of a feature of the required system. It does not mention finer details such as the different scenarios that can occur. Based on User stories, the project team proposes Metaphors. Metaphors are a common vision of how the system would work. The development team may decide to build a Spike for some features. A Spike is a very simple program that is constructed to explore the suitability of a solution being proposed. It can be considered similar to a prototype. Some of the basic activities

Some of the good practices that have been recognized in the extreme programming model and suggested to maximize their use are given below:



Fig2: Extreme Programming Good Practices

Code Review: Code review detects and corrects errors efficiently. It suggests pair programming as coding and reviewing of written code carried out by a pair of programmers who switch their work between them every hour. The concept of coding which is used in the XP model is slightly different from traditional coding. Here, the coding activity includes drawing diagrams (modeling) that will be transformed into code, scripting a web-based system, and choosing among several alternative solutions.

Testing: Testing code helps to remove errors and improves its reliability. XP suggests test-driven development (TDD) to continually write and execute test cases. In the TDD approach, test cases are written even before any code is written.

Incremental development: Incremental development is very good because customer feedback is gained and based on this development team comes up with new increments every few days after each iteration.

Simplicity: Simplicity makes it easier to develop good-quality code as well as to test and debug it. The main principle of the XP model is to develop a simple system that will work efficiently in the present time, rather than trying to build something that would take time and may never be used. It focuses on some specific features that are immediately needed, rather than engaging time and effort on speculations of future requirements.

Design: Good quality design is important to develop good quality software. So, everybody should design daily. Without a proper design, a system implementation becomes too complex, and very difficult to understand the solution, thus making maintenance expensive. A good design results in elimination of complex dependencies within a system. So, effective use of suitable design is emphasized.

Integration testing: Integration Testing helps to identify bugs at the interfaces of different functionalities. Extreme programming suggests that the developers should achieve continuous integration by building and performing integration testing several times a day.

Listening: The developers need to carefully listen to the customers if they have to develop good quality software. Sometimes programmers may not have the depth knowledge of the system to be developed. So, the programmers should understand the functionality of the system and they have to listen to the customers.

Feedback: One of the most important aspects of the XP model is to gain feedback to understand the exact customer needs. Frequent contact with the customer makes the development effective.

Pair Programming: XP encourages pair programming where two developers work together at the same workstation. This approach helps in knowledge sharing, reduces errors, and improves code quality.

Continuous Integration: In XP, developers integrate their code into a shared repository several times a day. This helps to detect and resolve integration issues early on in the development process.

Refactoring: XP encourages refactoring, which is the process of restructuring existing code to make it more efficient and maintainable. Refactoring helps to keep the codebase clean, organized, and easy to understand.

Collective Code Ownership: In XP, there is no individual ownership of code. Instead, the entire team is responsible for the codebase. This approach ensures that all team members have a sense of ownership and responsibility towards the code. This practice highlights the fact that the entire team owns the code. Each team member is responsible for reviewing and updating code. The implementation of collective code ownership encourages the team to cooperate more and be more open to bringing new ideas. No key player exists in this setup, so any developer can fix or handle any issue or bug.

Planning Game: XP follows a planning game, where the customer and the development team collaborate to prioritize and plan development tasks. This approach helps to ensure that the team is working on the most important features and delivers value to the customer.

On-site Customer: XP requires an on-site customer who works closely with the development team throughout the project. This approach helps to ensure that the customer's needs are understood and met, and also facilitates communication and feedback.

Metaphor: It is defined as the long-term visualization where every team member who is part of the project should have a high-level understanding of the overall system and coding requirements. This can be accomplished

by conducting brainstorming sessions related to the codebase, overall design, and architecture.

Coding standards and guidelines: Organizations want development teams to follow well-described and standard coding styles called coding standards. These are the standard guidelines followed in XP. It ensures that teams working on different modules or components can follow coding standards for maintaining consistency in the code, style, naming conventions, and use of different system parameters. Using fixed standards and guidelines, you can quickly identify and solve issues once the code is developed.

Continuous integration: The teams use different continuous integration (CI) tools, such as Jenkins, Azure DevOps Pipelines, etc. In this practice, the developers ensure that the system is fully integrated. XP teams work on an iterative development approach as they commit the code changes multiple times during the day, known as continuous delivery. Also, the developers discuss which parts of the code can be re-used or shared. It allows them to handle the code requirements effectively and understand the functionality they are trying to build. The shared code base eliminates integration issues.

40-hour week: XP projects require developers to be effective and fast in their product delivery, ensuring the quality aspect remains intact. To adhere to these requirements, they should be well-rested. To ensure that the professionals do not feel lethargic or overloaded with work, the total number of work hours should not exceed more than 40 per week.

Applications of Extreme Programming (XP)

Some of the projects that are suitable to develop using the XP model are given below:

Small projects: The XP model is very useful in small projects consisting of small teams as face-to-face meetings are easier to achieve.

Projects involving new technology or Research projects: This type of project faces changing requirements rapidly and technical problems. So the XP model is used to complete this type of project.

Web development projects: The XP model is well-suited for web development projects as the development process is iterative and requires frequent testing to ensure the system meets the requirements.

Collaborative projects: The XP model is useful for collaborative projects that require close collaboration between the development team and the customer.

Projects with tight deadlines: The XP model can be used in projects that have a tight deadline, as it emphasizes simplicity and iterative development.

Projects with rapidly changing requirements: The XP model is designed to handle rapidly changing requirements, making it suitable for projects where requirements may change frequently.

Projects where quality is a high priority: The XP model places a strong emphasis on testing and quality assurance, making it a suitable approach for projects where quality is a high priority.

XP, and other agile methods, are suitable for situations where the volume and space of requirements change are high and where requirement risks are considerable.

Life Cycle of Extreme Programming (XP)

The Extreme Programming Life Cycle consist of five phases:

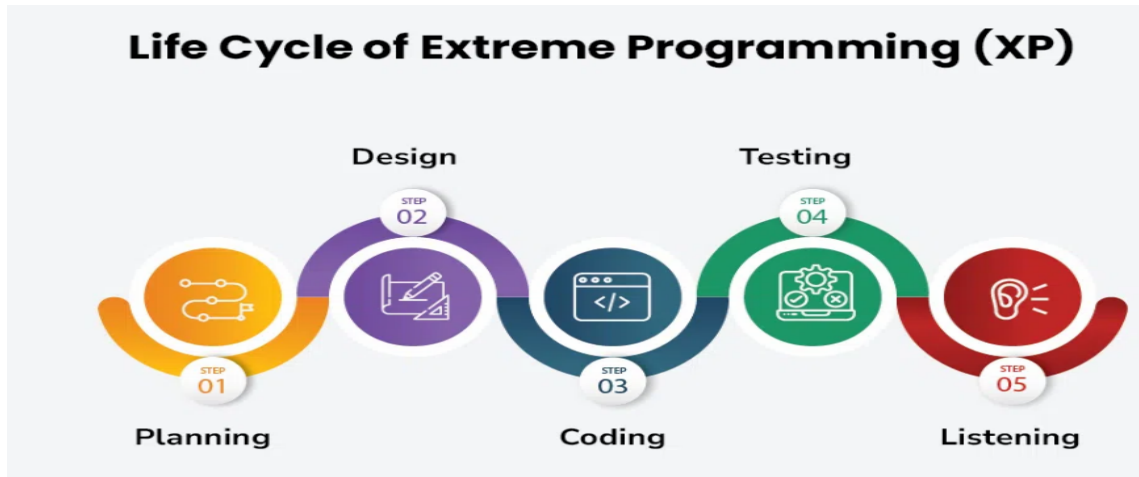


Fig3: Life Cycle of Extreme Programming (XP)

1. **Planning:** The first stage of Extreme Programming is planning. During this phase, clients define their needs in concise descriptions known as user stories. The team calculates the effort required for each story and schedules releases according to priority and effort.
2. **Design:** The team creates only the essential design needed for current user stories, using a common analogy or story to help everyone understand the overall system architecture and keep the design straightforward and clear.
3. **Coding:** Extreme Programming (XP) promotes pair programming i.e. two developers work together at one workstation, enhancing code quality and knowledge sharing. They write tests before coding to ensure functionality from the start (TDD), and frequently integrate their code into a shared repository with automated tests to catch issues early.
4. **Testing:** Extreme Programming (XP) gives more importance to testing that consists of both unit tests and acceptance tests. Unit tests, which are automated, check if specific features work correctly. Acceptance tests, conducted by customers, ensure that the overall system meets initial

- requirements. This continuous testing ensures the software's quality and alignment with customer needs.
5. **Listening:** In the listening phase regular feedback from customers to ensure the product meets their needs and to adapt to any changes.

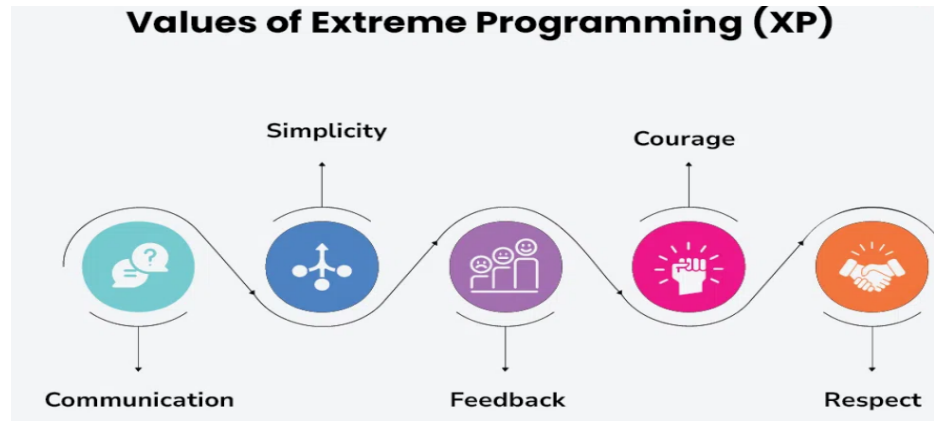


Fig4: Values of Extreme Programming (XP)

1. **Communication:** The essence of communication is for information and ideas to be exchanged amongst development team members so that everyone has an understanding of the system requirements and goals. Extreme Programming (XP) supports this by allowing open and frequent communication between members of a team.
2. **Simplicity:** Keeping things as simple as possible helps reduce complexity and makes it easier to understand and maintain the code.
3. **Feedback:** Feedback loops which are constant are among testing as well as customer involvements which helps in detecting problems earlier during development.
4. **Courage:** Team members are encouraged to take risks, speak up about problems, and adapt to change without fear of repercussions.
5. **Respect:** Every member's input or opinion is appreciated which promotes a collective way of working among people who are supportive within a certain group.

Advantages of Extreme Programming (XP)

Slipped schedules: Timely delivery is ensured through slipping timetables and doable development cycles.

Misunderstanding the business and/or domain – Constant contact and explanations are ensured by including the client on the team.

Canceled projects: Focusing on ongoing customer engagement guarantees open communication with the consumer and prompt problem-solving.

Staff turnover: Teamwork that is focused on cooperation provides excitement and goodwill. Team spirit is fostered by multidisciplinary cohesion.

Costs incurred in changes: Extensive and continuing testing ensures that the modifications do not impair the functioning of the system. A functioning system always guarantees that there is enough time to accommodate changes without impairing ongoing operations.

Business changes: Changes are accepted at any moment since they are seen to be inevitable.

Production and post-delivery defects: the unit tests to find and repair bugs as soon as possible.

Disadvantages of Extreme programming

- In many scenarios, customers are not sure of the exact requirement, which makes it challenging to define scope, cost, and time accurately.
- XP relies too heavily on programming principles, where the design is often overlooked. This can be challenging when your product contains a lot of design changes that make a difference in the end-user experience.
- There are regular meetings with customers, which often take a great deal of time that could be spent on writing the code.
- The rapid transition from the traditional approach to XP demands a lot of cultural and structural changes.
- Sometimes, customers do not have the time or expertise to participate in product development. If we have strict timelines, it becomes a problem as valuable feedback cannot be provided on time, impacting the overall quality.

Extreme programming (XP) vs. Waterfall methodology

Extreme programming	Waterfall methodology
It is an iterative process that runs in the same lifecycle repeatedly until the final product is created.	It involves breaking down project activities into linear sequential phases, each of which is dependent on the previous one and categorized accordingly.
The planning begins at the start of the development cycle.	There is a dedicated planning phase that is independent of the development cycle.
The testing is performed before the actual code is implemented.	The testing is performed once the development requirements are completed.
This approach works when the project requirements change in every iteration.	This approach works when the project requirements and overall scope is fixed.

Applying XP in Practice : XP is ideally suited for projects with:

- **Small teams:** XP practices and values work effectively for smaller teams (less than 12 people). It becomes easier to manage small teams as communication is seamless, and it takes less time to conduct meetings and code review sessions.
- **Automated tests:** The ability of the development teams to create and perform automation testing using standard tools and frameworks.
- **Active customer participation:** XP is all about customers' active participation with different project stakeholders. The quality output is ensured if the customer is always available to provide inputs until the project is executed.
- **Ready to accept a new culture and mindset:** XP is entirely different from the traditional software development approach. It becomes important for the organization and associated teams to embrace changes. It will also require changing the overall mindset and ways of working.

- **Highly adaptive development approach:** XP works perfectly for systems with frequent changes. The framework is designed to help development teams adapt to changing requirements quickly and perfectly.
- **Reduced risk:** The iterative nature of XP allows for early detection and correction of errors. Small releases minimize the impact of potential problems and make it easier to recover from setbacks.
- **Improved software quality:** XP's emphasis on testing, continuous integration, and code reviews leads to a more robust and maintainable product.
- **Faster time to market:** Frequent releases enable early feedback and faster delivery of value to customers. This allows for course correction and reduces the risk of building features that nobody wants.
- **Increased team morale:** Collaboration and communication are central to XP, fostering a more engaged and motivated development team. Shared ownership and continuous improvement create a sense of accomplishment and purpose.
- **High project visibility:** XP thrives on transparency and open communication, making it a good fit for projects with clear stakeholder involvement.

Here's an example of how XP Extreme Programming might be applied in practice:

Project: Developing a new mobile learning application.

Process: The development team works in short iterations (e.g., two weeks). At the beginning of each iteration, they hold a planning game with the on-site customer representative to prioritize user stories, which could include features like content creation, quiz functionalities, and progress tracking.

Development: Developers practice pair programming and TDD to build the functionalities outlined in the user stories. Throughout the iteration, there's continuous integration and automated testing to ensure quality. Daily stand-up meetings keep everyone informed of progress and potential roadblocks.

Delivery: At the end of the iteration, a working version of the app with the completed user stories is deployed for customer feedback. This feedback then informs the priorities for the next iteration. The cycle continues until the core functionalities of the app are complete, with ongoing iterations for further refinement and feature development.

While the core XP practices are well-established, XP is also an adaptable framework. Here are some additional practices that some teams may find beneficial:

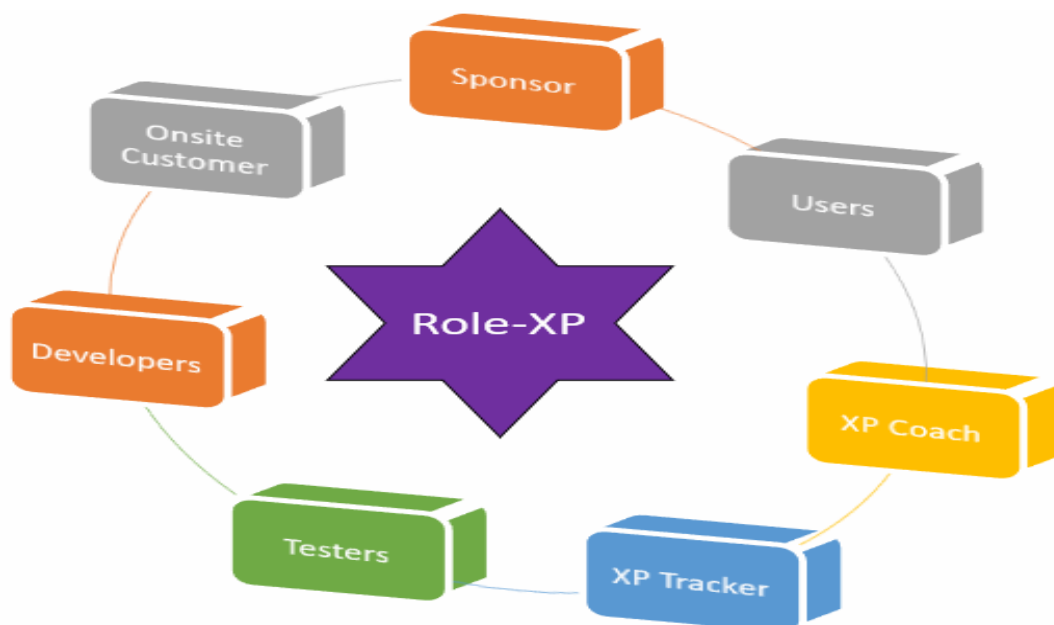
- **Collective Code Ownership:** The entire team takes responsibility for the codebase, fostering a sense of shared accountability and encouraging code reviews and refactoring.
- **Spike Solutions:** For particularly complex or risky tasks, short, focused efforts (spikes) can be undertaken to gather information and reduce uncertainty before diving into full development.
- **Coding Standards:** Establishing clear coding standards promotes consistency, readability, and maintainability of the codebase. This can be especially helpful in pair programming environments.
- **Continuous Learning:** XP encourages a culture of continuous learning for developers. This can involve attending workshops, conferences, or participating in code reviews to stay up-to-date with the latest technologies and best practices.

Benefits and Considerations of XP:

- **Learning curve:** Implementing XP extreme Programming practices effectively may require a cultural shift for teams accustomed to traditional development methodologies. Training and mentorship can help ease the transition.
- **Overhead:** Practices like pair programming and continuous integration can initially seem time-consuming. However, the benefits in terms of quality, knowledge sharing, and early problem detection often outweigh the cost.
- **Scalability:** XP may be less suitable for very large teams or projects with highly complex requirements. The intensive communication and collaboration practices can become challenging to manage in large-scale projects.

The XP Team, XP Concepts

Like Scrum roles, XP also has a cross-functional team that works on stories and releases that at the end of each Iteration. This article introduces some of the roles seen in XP teams.



- **The Whole Team:** Like the Scrum team, the XP Team is also self-organized, cross-functional, and co-located and also does ceremonies like planning

meetings, daily standup meetings, review meetings, and retrospective meetings in each iteration. An XP team can consist of 5 to 20 members having various and complementary skills. In XP it is not recommended to work on multiple work items or projects, as context switching of developers will hamper the valuable work.

- **XP Coach:** Since the team is self-organized and cross-functional, a management layer or explicit leadership role is often found not useful. Instead of traditional management roles, the XP Coach plays a supporting role in the team's success. A coach helps a team follow the process and helps the team to learn discipline and make sure that XP Principles are followed well by the team. They set up a work culture that's balanced with the rest of the organization, mentor the team to contribute to their tasks, facilitate the team, and help them perform.
- **On-Site Customers:** As the Product Owner role in the scrum, in XP - the On-Site Customer role is responsible for maximizing the product value and defining the features and stories with acceptance criteria. They are domain or product specialists and experts in release planning crafting project vision, maximizing the return on investment in each iteration, mitigating risks, and providing feedback to the team for the work done at the end of every release. If the on-site customer is not present at that time role can be filled with product owners, product managers, UI designers, and business analysts as proxies roles. The word "on-site" means that the customers or their proxies sit together with the team to ensure that communication flows smoothly.
- **Programmer:** The XP team has 4-10 developers who can work together practicing XP core values like pair programming, practice test-driven development, continuous integration, automation testing, and reducing technical debt by refactoring the code as required. Each team should have at least one designer, senior programmer, and system architect so that the team can work toward an incremental design and architecture. They pull the user stories and work in a self-organized way to achieve the iteration goal.
- **Testers:** Tester is also one of the important roles and a rule of thumb is to have one tester per four programmers. In some cases, XP team members play the role of testers and there aren't any dedicated testers, the programmers and the on-site customers are expected to fill the role by using

strategies like Test-Driven Development, automated regression testing, or exploratory testing. In general, Testers assist the on-site customer in writing the acceptance test criteria, executing test cases, and communicating their results back to the programmers. Unlike traditional methodologies, XP testers do not perform long manual regression cycles. Instead, they use techniques like exploratory testing to help the team identify whether it is successfully preventing bugs from reaching finished code. Testers also look at nonfunctional requirements like usability, performance, stability, resilience, and ability to handle long periods of load and stress.

- **XP Tracker:** XP Tracker is the role to keep track of the team's progress at an iteration level. The Tracker tracks the progress of the team, collects the information and metrics from the tools information, and prepares the reports without disturbing the team. Sometimes the tracker can also facilitate activities within the team and help them to do external communication.
- **Sponsor:** The sponsor is the person who is funding the project and is a very important stakeholder of the project. A team must interact with sponsors regularly to provide demos and get valuable feedback. Above are the core roles in XP but some of the other roles might be relevant and important based on the situation like Business Analyst(BA), UI designer, System architect, and domain expert. Extreme Programming (XP) is an agile software development methodology designed to deliver high-quality software through continuous feedback, collaboration, and adaptation. It's a specific and prescriptive approach within the broader agile umbrella, emphasizing a set of core values, principles, and practices.

Core Concepts of XP (Extreme Programming)

- **Values:** Communication, simplicity, feedback, courage, and respect form the foundation of XP. These values guide decision-making and team interactions throughout the development process.
- **Communication:** XP thrives on open and transparent communication between developers, customers, and stakeholders. Daily stand-up meetings and user story creation sessions ensure everyone is on the same page.

- **Simplicity:** XP promotes a focus on building the simplest solution that meets the current need. This avoids over-engineering and keeps the codebase maintainable.
- **Feedback:** Continuous feedback is crucial in XP. Techniques like pair programming and on-site customers ensure constant evaluation and improvement of the developing software.
- **Courage:** XP encourages developers to be brave in refactoring code, experimenting with new ideas, and challenging assumptions. This fosters a culture of continuous learning and improvement.
- **Respect:** Mutual respect between team members, customers, and stakeholders is essential for effective collaboration and a positive development environment.
- **Principles:** XP translates its values into actionable principles. Examples include:
 - **Small Releases:** Frequent deployments of working software in short cycles enable rapid feedback and course correction. This reduces risk and allows for early validation of ideas.
 - **Embrace Change:** XP acknowledges that requirements can evolve throughout a project. The iterative nature of XP allows for continuous adaptation to changing needs without derailing the project.
 - **Work in Short Iterations:** Breaking down development into small, time-boxed iterations (e.g., one to two weeks) promotes focus and keeps the project on track. Each iteration delivers a potentially shippable product increment.
 - **Sustainable Pace:** XP emphasizes creating a development process that developers can maintain over time. This includes practices like pair programming, which can reduce burnout and improve code quality.

Practices: These are the concrete methods that put XP's principles into action. Some of the most well-known XP practices include:

- **Pair Programming:** Two developers work together on a single task, fostering knowledge sharing and improving code quality through real-time code review and collaboration.
- **Test-Driven Development (TDD):** Writing unit tests before code ensures functionality and maintainability. This practice emphasizes clear and concise code that meets specific requirements.
- **Continuous Integration:** Frequent code merges and automated builds minimize integration issues and ensure the entire codebase remains functional at all times.
- **On-Site Customer:** A dedicated customer representative actively participates in the development process, providing continuous feedback on priorities and functionality. This ensures the software remains aligned with actual user needs.
- **Planning Game:** This collaborative session brings together developers and customers to define priorities, estimate effort, and create a plan for the next iteration.
- **Refactoring:** Regular code refactoring improves code structure and maintainability without altering functionality. This helps keep the codebase clean and adaptable as the project evolves.

Case Studies for Extreme programming

The following are some case studies illustrating how XP practices changed project delivery dynamics and quality.

- XP for Krizp System

Krizp System was a startup providing web development services in India. They used to create web portals for other small companies or educational institutions. They had no defined framework for software development processes as it was a startup with few employees and limited resources. The requirement-gathering process was not structured, and it became a lot more challenging to handle the requirements when the customer proposed changes to the existing requirements that were delivered.

The developers were not communicating frequently as they worked independently on different modules or components. There was no project manager to track the progress and health of the project or even make sure that there were regular interactions with customers so that the product requirements were developed as per their expectations.

The team at Krizp system was introduced to the concepts related to different Agile frameworks, and this is when XP methodology was introduced for one month. The company's CEO took up two roles – customer representative and tracker. This meant that the user stories prioritization, regular customer interaction, and handling development teams seamlessly.

The pair programming technique was adopted when developers were working together. The design improvements were made based on customer feedback and inputs. The concept of XP positively impacted the company's overall software development cycle, with better team productivity and effective product deliveries.

XP for IBM and Sabre Airlines

The researchers at IBM wanted to study the impact of XP on different crucial aspects such as productivity, customer satisfaction, etc. There were two different

approaches – waterfall and XP for developing their applications. The final results indicated that XP is a far better approach than waterfall.

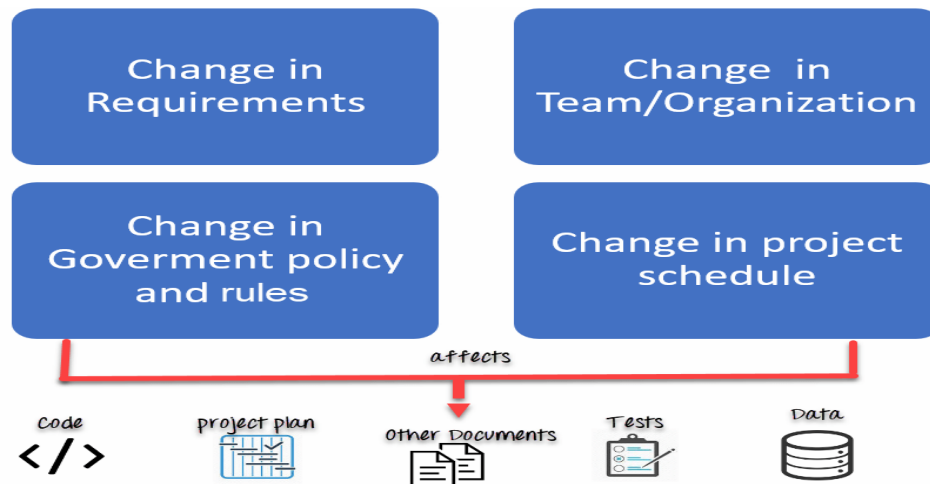
In another case study at Sabre, Airline Solutions followed two approaches to release the same product. Delivering the release took 3.5 months using XP compared to the waterfall, which took 18 months. This shows how XP creates a larger business and customer impact from an output perspective.

Software Configuration Management(SCM)

SCM is a process to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle. The primary goal is to increase productivity with minimal mistakes. SCM is part of the cross-disciplinary field of configuration management and it can accurately determine who made which revision.

The primary reasons for Implementing Technical Software Configuration Management System are:

- There are multiple people working on software which is continually updating
- It may be a case where multiple version, branches, authors are involved in a software config project, and the team is geographically distributed and works concurrently
- Changes in user requirement, policy, budget, schedule need to be accommodated.
- Software should be able to run on various machines and Operating Systems
- Helps to develop coordination among stakeholders
- The SCM process is also beneficial to control the costs involved in making changes to a system.



Any change in the software configuration Items will affect the final product. Therefore, changes to configuration items need to be controlled and managed.

Tasks in SCM process

- Configuration Identification
- Baselines
- Change Control
- Configuration Status Accounting
- Configuration Audits and Reviews

Configuration Identification

Configuration identification is a method of determining the scope of the software system. With the help of this step, you can manage or control something even if you don't know what it is. It is a description that contains the CSCI type (Computer Software Configuration Item), a project identifier and version information.

Activities during this process:

- Identification of configuration Items like source code modules, test case, and requirements specification.
- Identification of each CSCI in the SCM repository, by using an object-oriented approach

- The process starts with basic objects which are grouped into aggregate objects. Details of what, why, when and by whom changes in the test are made
- Every object has its own features that identify its name that is explicit to all other objects
- List of resources required such as the document, the file, tools, etc.

Example:

- Instead of naming a File login.php its should be named login_v1.2.php where v1.2 stands for the version number of the file
- Instead of naming folder “Code” it should be named “Code_D” where D represents code that should be backed up daily.

Baseline: A baseline is a formally accepted version of a software configuration item. It is designated and fixed at a specific time while conducting the SCM process. It can only be changed through formal change control procedures.

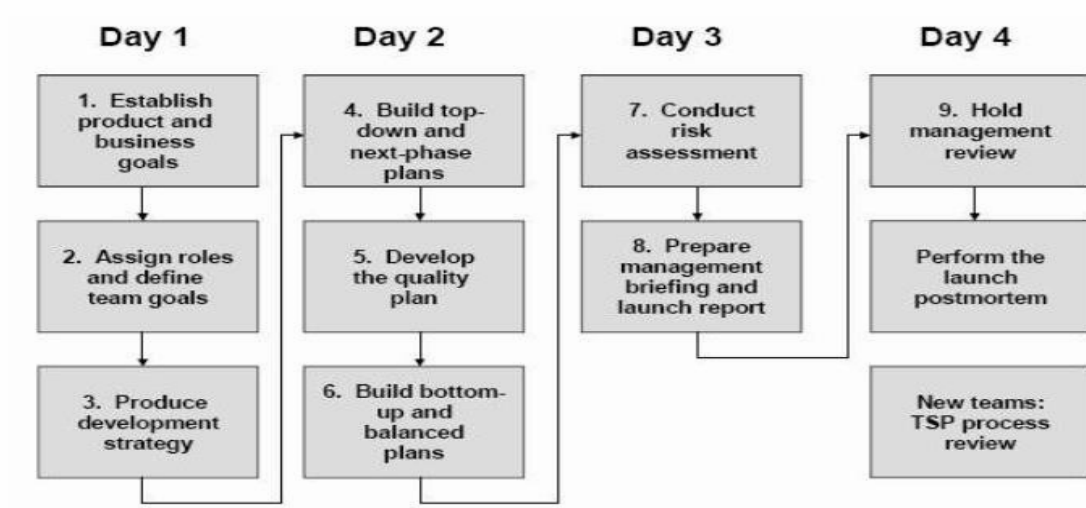
Activities during this process:

- Facilitate construction of various versions of an application
- Defining and determining mechanisms for managing various versions of these work products
- The functional baseline corresponds to the reviewed system requirements
- Widely used baselines include functional, developmental, and product baselines

The baseline process involves the following;

- 1) A series of software engineering tasks produces a CSCI.
- 2) The CSCI is reviewed and possibly approved.
- 3) The approved CSCI is given a new version number and placed in a project database (i.e., software repository).
- 4) A copy of the CSCI is taken from the project database and examined/modified by a software engineer.
- 5) The baseline of the modified CSCI goes back to Step #2.

Automated SCM Repository



Functions of an SCM Repository

- Data integrity – It validates entries, ensures consistency, cascades modifications.
- Information sharing – It shares information among developers and tools, manages and controls multi-user access.
- Tool integration – It establishes a data model that can be accessed by many software engineering tools, controls access to the data.
- Data integration – It allows various SCM tasks to be performed on one or more CSCIs.
- Methodology enforcement – It defines an entity-relationship model for the repository that implies a specific process model for software engineering.
- Document standardization – It defines objects in the repository to guarantee a standard approach for creation of software engineering documents.

In simple words, baseline means ready for release.

Change Control **Change Management and Control**

Change control is a procedural method which ensures quality and consistency when changes are made in the configuration object. In this step, the change request is submitted to the software configuration manager.

Activities during this process:

- Control ad-hoc change to build a stable software development environment. Changes are committed to the repository
- The request will be checked based on the technical merit, possible side effects and overall impact on other configuration objects.
- It manages changes and making configuration items available during the software lifecycle

Configuration Status Accounting: Configuration status accounting tracks each release during the SCM process. This stage involves tracking what each version has and the changes that lead to this version.

Activities during this process:

- Keeps a record of all the changes made to the previous baseline to reach a new baseline
- Identify all items to define the software configuration
- Monitor status of change requests
- Complete listing of all changes since the last baseline
- Allows tracking of progress to next baseline
- Allows to check previous releases/versions to be extracted for testing

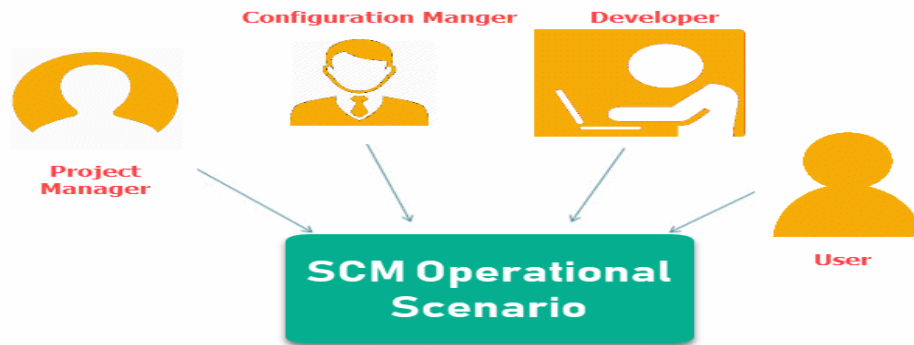
Configuration Audits and Reviews: Software Configuration audits verify that all the software product satisfies the baseline needs. It ensures that what is built is what is delivered.

Activities during this process:

- Configuration auditing is conducted by auditors by checking that defined processes are being followed and ensuring that the SCM goals are satisfied.
- To verify compliance with configuration control standards. auditing and reporting the changes made
- SCM audits also ensure that traceability is maintained during the process.
- Ensures that changes made to a baseline comply with the configuration status reports
- Validation of completeness and consistency

Participant of SCM process

Following are the key participants in SCM



1. Configuration Manager

- Configuration Manager is the head who is Responsible for identifying configuration items.
- CM ensures team follows the SCM process
- He/She needs to approve or reject change requests

2. Developer

- The developer needs to change the code as per standard development activities or change requests. He is responsible for maintaining configuration of code.
- The developer should check the changes and resolves conflicts

3. Auditor

- The auditor is responsible for SCM audits and reviews.
- Need to ensure the consistency and completeness of release.

4. Project Manager:

- Ensure that the product is developed within a certain time frame
- Monitors the progress of development and recognizes issues in the SCM process
- Generate reports about the status of the software system
- Make sure that processes and policies are followed for creating, changing, and testing

5. User

The end user should understand the key SCM terms to ensure he has the latest version of the software

Software Configuration Management Plan

The SCMP (Software Configuration management planning) process planning begins at the early coding phases of a project. The outcome of the planning phase is the SCM plan which might be stretched or revised during the project.

- The SCMP can follow a public standard like the IEEE 828 or organization specific standard
- It defines the types of documents to be managed and a document naming. Example Test_v1
- SCMP defines the person who will be responsible for the entire SCM process and creation of baselines.
- Fix policies for version management & change control
- Define tools which can be used during the SCM process
- Configuration management database for recording configuration information.

Software Configuration Management Tools

Any Change management software should have the following 3 Key features:

- **Concurrency Management:** When two or more tasks are happening at the same time, it is known as concurrent operation. Concurrency in context to SCM means that the same file being edited by multiple persons at the same time. If concurrency is not managed correctly with SCM tools, then it may create many pressing issues.
- **Version Control:** SCM uses archiving method or saves every change made to file. With the help of archiving or save features, it is possible to roll back to the previous version in case of issues.
- **Synchronization:** Users can check out more than one file or an entire copy of the repository. The user then works on the needed file and checks in the

changes back to the repository. They can synchronize their local copy to stay updated with the changes made by other team members.

Following are popular tools

1. Git: Git is a free and open source tool which helps version control. It is designed to handle all types of projects with speed and efficiency.

2. Team Foundation Server: Team Foundation is a group of tools and technologies that enable the team to collaborate and coordinate for building a product.

3. Ansible: It is an open source Software configuration management tool. Apart from configuration management it also offers application deployment & task automation.

- Configuration Management best practices help organizations to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle.
- The primary goal of the SCM process is to increase productivity with minimal mistakes
- The main reason behind the configuration management process is that there are multiple people working on software which is continually updating. SCM helps establish concurrency, synchronization, and version control.
- A baseline is a formally accepted version of a software configuration item
- Change control is a procedural method which ensures quality and consistency when changes are made in the configuration object.
- Configuration status accounting tracks each release during the SCM process
- Software Configuration audits verify that all the software product satisfies the baseline needs
- Project manager, Configuration manager, Developer, Auditor, and user are participants in SCM process
- The SCM process planning begins at the early phases of a project.
- Git, Team foundation Server and Ansible are few popular SCM tools.

Version control systems (VCS)

VCS give software engineering teams complete visibility to the code history and a single source of documentation for all files, folders, and messages. Version control tools streamline software development and mitigate lost work and time by tracking code changes from asynchronous and concurrent work, identifying conflicting edits, sparking collaboration, and preventing overwrites. Version control allows the developer "orchestra" to see every commit and access, review, collaborate, experiment, compare, and undo changes to ensure code integrity and faster releases.

Version control software helps facilitate continuous software development workflows. As user demands scale up, version control helps developers work smarter together, using time and resources more efficiently.

A foundational tool in the modern developer's toolkit, version control tools keep a historical record of software changes in a specialized database, logging edits made by individual developers. When conflicts emerge, developers can look back and resolve code conflicts, minimizing disruption to the codebase. Especially useful for DevOps and DevSecOps teams in accelerated cloud-based environments, version control systems facilitate collaboration, productivity, and successful software deployments.

Understanding version control systems

Version control systems help eliminate common development roadblocks—like operating system limitations and siloed tool chains—simplifying and streamlining development and creating space for innovation that can lead to breakthroughs. In addition to accelerating productivity and reducing errors, version control systems help save enterprises time and money.

Version control is essential to the overall health of the software development lifecycle (SDLC) and improves project management and product development. In fact, failure to adopt version control poses risks like data loss, sluggish development, and reduced code quality, hampering competitiveness. Integrating version control mitigates these risks by helping enterprises scale processes as demand increases. A secure, detailed record of versions and releases creates a hedge against code corruption.

Using a version control tool helps an organization unify operating systems, services, and developer toolkits by tearing down siloes and facilitating coordination across the entire software engineering ecosystem. Greater visibility fosters better communication and infuses buoyancy into the development process by improving project clarity and keeping contributors engaged and aligned.

Version control systems empower developer teams to:

- Create a codebase history: Version control systems create a complete codebase history, stored in a specialized database, and provide the entire team with a single and secure source of truth.
- Ramp up collaboration: By tracking every change, version control systems help teams avoid conflict and create opportunities to experiment and innovate.
- Reduce errors: Using a version control tool allows developers to find errors fast, roll back to a previous version and correct the problem, mitigating the impact of the error.
- Improve code quality: A version control system encourages developers to follow best practices and write clean code that is easy to access, understand, and maintain.
- Recover in a snap: With version control, every code change is tracked, restorable, and revertible. A version control system acts as a safety net, preventing accidental code deletions and related developer anxiety.
- Increase coding confidence: Developers rely on version control systems to synchronize versions so they can resolve conflicts and safely experiment.
- Expand visibility: Team collaboration and communication improve with version control by providing full visibility to the code documentation and its history.
- Automate tasks: Efficiency and productivity increase with a version control system by automating testing, analysis, and deployment for fast, consistent results.

Features of version control software

Let's review some version control software's capabilities, functions, and features.

- Repository: Also called a "repo," a repository is the centralized database that stores the complete collection of files and folders for a codebase, along with the revision history.
- Pull request: The mechanism developers use to propose, notate, review, and discuss changes before they merge updates into the main codebase is a pull request. A pull request is also known as a merge request.
- Commit: A commit is a snapshot of changes with a unique "hash" that identifies the proposed changes. A commit can include notes and messages between developers.
- Branch: A code branch is a separate, parallel version of the codebase created by developers to work independently on experiments, regression testing, and debugging without changing the main codebase.
- Merge: When developers combine code edits, they integrate the changes from one branch into another or into the main codebase.
- Conflict: When multiple developers make edits to the code, their changes sometimes conflict. Version control tools help developers identify and resolve conflicts to keep development moving.
- Checkout: When a developer retrieves a file from the version control system it's called a checkout.
- Tag: A tag is a marker used by contributors to label a specific point in the source code history, like the release date. Tags are also used to mark a specific point in the codebase before changes.
- Remote: Remote development allows developers to do some or all their work on their local desktop, on a company server, or on the cloud.
- Fork: A fork is the process of creating a separate and distinct piece of software by copying source code from an existing software package.
- Revert: Developers can revert, or undo, one or more recent changes and return to the previous version.

Types of version control systems

Deciding which tool is right depends on the needs of the development team, scale of the project, and other factors, but having one is essential to efficient and effective software development.

Some of the distinct types of version control systems are:

1. Local

- Stores code changes locally on a user's computer
- File changes are stored as patches

- Patches are pushed to a single version of the codebase
- Not scalable or collaborative, works for smaller projects

2. Centralized version control systems (CVCS)

- A single, centralized database, or repository, is stored on a server
- All users work with the same repository and commit to the same branch
- Users check out the latest version from the server
- Some systems "file lock" in the central repository to prevent simultaneous edits to the same file
- Contributors push commits to the server and resolve merge conflicts on the repository.

3. Distributed version control systems (DVCS)

- Users access the repository from any location
- Contributors can work on the same codebase without being on the same network
- Each developer has a local copy of the entire repository and history on their device
- Users can commit, branch, and merge changes locally without reliance on the central server.

4. Lock-based

- When a user begins work, individual files are locked.
- Prevents two or more users from making conflicting changes.

5. Optimistic

- Every user has a private workspace
- Users submit server requests to share changes with the team before merging
- The server determines which changes can be merged safely

Depending on region and locale, distributed and centralized version control systems are the most widely used types because of their scalability, flexibility, and developers can collaborate on code from anywhere in the world.

Common version control tools

There are several DevOps tools available for version control, and the most used include:

- Git: Git is an open-source distributed version control tool preferred by developers for its speed, flexibility, and because contributors can work on the same codebase simultaneously.
- Subversion (SVN): Subversion is a centralized version control tool used by enterprise teams and is known for its speed and scalability.
- Azure DevOps Server: Previously known as Microsoft Team Foundation Server (TFS), Azure DevOps Server is a set of modern development services, a centralized version control, and reporting system hosted on-premises.
- Mercurial: Like Git in scalability and flexibility, Mercurial is a distributed version control system.
- Perforce: Used in large-scale software development projects, Perforce is a centralized version control system valued for its simplicity and ease of use.

Version Control tool factors

When evaluating the advantages and disadvantages of various version control tools, keep these factors in mind.

- Scalability: If the project is large with a distributed team, the tool should be able to handle expansive projects with many files and users.
- Ease of use: Version control tools can be complex, so a simple, user-friendly interface can help ensure a manageable learning curve and accelerate adoption.
- Collaboration features: The tool should support multiple contributors working on the same codebase simultaneously and facilitate quick communication to streamline merges.
- Integration with existing tools: Look for a tool that will work with the tools your team already uses, like project management software, continuous software integration solutions, building and editing tools, and integrated development environments.
- Supports branching: A VCS should support branching for efficient workflows so developers can work on distinct parts of the code without conflicting or interfering with one another's work.

Best practices for version control

Incorporating version control into a software development process is a significant step. After choosing the right version control tool for a project, best practices can help the team get up to speed early.

- The repository setup: A version control repository should be compliant, and have access control, disaster recovery, and failover plans in place.
- Establish workflow alignment: Productive collaboration is optimized when contributors are on the same workflow page, so seek understanding and agreement early.
- Write clear commit messages: When contributors commit to the repository, including clearly written messages that explain the changes can help mitigate roadblocks.
- Commit often: Encourage the team to make lots of small commits with concise comments to keep development moving. Avoid the tendency to make fewer but more complex changes accompanied by long messages.
- Test before committing code: The adage, "Measure twice, cut once," applies here. To ensure the code works as expected, contributors should test before committing changes to the codebase.
- Commit code only when ready: Like doing many small commits, only completed code should be committed to keep the codebase in good working order.
- Avoid conflicts: Developers can help mitigate conflicts by pulling code from upstream to ensure they work with the most up to date code, and by breaking large files into smaller ones.
- Use branches: Code branches should support multiple versions of software releases and patches.
- Limit repository access: Give access only to contributors who need it, a simple but powerful practice that prevents unauthorized codebase access.